



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Entrenamiento en entorno virtual para la teleoperación segura de Brazos Robóticos

Realidad Virtual – Facultad de Ingeniería



Profesores

Ing. Javier Rosenstein

Ing. Carlos Tomba

Alumnos

Quiroga, Juan Ignacio – 13889

Castel, Francisco – 13784

Índice

1	Introducción y Motivación	2
1.1	Descripción del problema	2
1.2	Motivación para el entorno de Realidad Virtual	3
2	Objetivos y Alcance	3
3	Hardware de interfaz: Joystick RPi Pico	4
4	Desarrollo en Unity	6
4.1	Listado de Comandos y Canvas	7
4.2	Remapeo Geométrico de Direcciones	9
4.3	Control PID en Articulaciones	10
4.4	Movimiento del Gripper y Realimentación Háptica	11
4.5	Comunicación UDP con Arquitectura Servidor–Cliente	12

1 Introducción y Motivación

1.1 Descripción del problema

La empresa **AUTOELEC** se dedica al asesoramiento, venta, reparación y alquiler de autoelevadores y baterías. Dentro de su proceso productivo, una tarea crítica es el ensamble de paquetes de baterías de plomo-ácido destinadas a autoelevadores industriales. Dichos paquetes se construyen ubicando manualmente múltiples vasos (celdas individuales) de entre 20 y 100 kg dentro de un rack metálico, conformando la batería final que alimenta el vehículo.



Figura 1: Vastos de plomo-ácido dispuestos en rack previo al ensamble.



Figura 2: Paquete de baterías terminado, marca Eternity – distribuido por AUTOELEC.

Al tratarse de un proceso completamente manual, el operador debe elevar cada vaso asistido únicamente por su propia fuerza física y ubicarlo en una posición determinada dentro del rack. A medida que el rack se va completando, el espacio libre se reduce notablemente, lo que obliga al operador a trabajar en posiciones forzadas durante la inserción de las últimas unidades. Esta combinación de esfuerzo físico considerable, movimientos repetitivos y posturas comprometidas genera riesgos ergonómicos que provocan un desgaste prematuro en la salud del personal, además de representar un riesgo concreto de accidentes durante la operación.

1.2 Motivación para el entorno de Realidad Virtual

La teleoperación y el aprendizaje de sistemas de manipulación robótica presentan desafíos particulares en contextos industriales. Durante la fase de entrenamiento existe riesgo de colisiones con el rack, caída de vasos y golpes al operador; al mismo tiempo, la tarea exige una precisión manual considerable en el posicionamiento de piezas pesadas, y la destreza inicial varía significativamente entre distintos operadores.

Un entorno de **Realidad Virtual (VR)** permite abordar estos desafíos de forma integral. En primer lugar, el operador aprende a manejar el manipulador sin riesgo de dañar equipos ni lastimarse, al interactuar con una réplica virtual del sistema real. Esto elimina a su vez los costos asociados a errores durante el entrenamiento, como materiales dañados o paradas de producción no planificadas. Además, el entorno virtual puede reconfigurarse libremente para distintas secuencias de ensamble, cargas o restricciones sin ninguna intervención física. Desde el punto de vista de la evaluación del desempeño, la plataforma permite instrumentar automáticamente indicadores objetivos como el tiempo por tarea, la suavidad del movimiento, la cantidad de colisiones y los reintentos necesarios. Finalmente, el joystick físico utilizado durante el entrenamiento virtual es el mismo con el que se opera el robot real, lo que facilita la transferencia directa de los patrones de movimiento aprendidos en VR al control del equipo físico.

En este contexto, el presente proyecto propone el desarrollo de un entorno VR para entrenar operarios en la teleoperación de un brazo robótico industrial, utilizando un joystick diseñado ad hoc sobre *Raspberry Pi Pico* como interfaz de control y el motor gráfico *Unity* como plataforma de simulación e interacción.

2 Objetivos y Alcance

El objetivo general del proyecto es desarrollar un entorno VR y un esquema de control que permitan entrenar y luego teleoperar un brazo robótico mediante un joystick propio, priorizando seguridad, usabilidad y desempeño. Para lograrlo, el trabajo se organizó en torno a un conjunto de objetivos específicos que abarcaron tanto el software como el hardware del sistema.

En el plano del software, se buscó implementar un simulador del brazo robótico en Unity con interacción XR, diseñar el mapeo de los controles del joystick a los grados de libertad del robot, e incorporar mecanismos de seguridad como parada de emergencia, zonas prohibidas y límites de velocidad. Complementariamente, se desarrollaron herramientas de monitoreo externo mediante comunicación UDP, que permiten visualizar el estado articular del robot desde distintos dispositivos en tiempo real. En el plano del hardware, el objetivo fue construir un prototipo funcional del joystick RPi Pico que opera como dispositivo HID estándar, incluyendo la capacidad de emitir

retroalimentación háptica hacia el operador.

El alcance del proyecto comprende el entrenamiento VR de tareas representativas de la aplicación industrial —en particular, el posicionamiento preciso del efector final sobre las celdas de batería—, la teleoperación con el joystick RPi Pico mediante comunicación USB, la integración de la capa de comunicación UDP para clientes externos y la implementación de una capa básica de seguridad en la simulación. Quedan fuera del alcance de esta versión la retroalimentación háptica proporcional a la fuerza de contacto, el *motion planning* autónomo, la integración física directa con el robot electrohidráulico real —que se reserva como trabajo futuro— y cualquier certificación normativa industrial.

3 Hardware de interfaz: Joystick RPi Pico

Se diseñó y construyó un controlador de dos ejes analógicos con carcasa ergonómica impresa en 3D, cuyo núcleo es un microcontrolador **Raspberry Pi Pico**. El diseño aprovecha los conversores analógico-digitales (ADC) integrados en el microcontrolador para la lectura de los potenciómetros de cada eje, y su interfaz USB nativa para la comunicación con el host. La carcasa fue modelada e impresa en PLA, adoptando una forma bimanual que permite sostener el dispositivo de forma natural durante sesiones prolongadas.

El firmware fue desarrollado en C/C++ sobre el SDK oficial de RPi Pico. Para la capa de comunicación se implementó el protocolo **USB HID** utilizando la librería *TinyUSB*, lo que permite que el joystick sea reconocido automáticamente por el sistema operativo como un gamepad estándar sin necesidad de drivers adicionales. Esta decisión simplifica significativamente la integración con Unity: el *Input System* del motor detecta el dispositivo y expone sus ejes y botones directamente en el editor, sin código de inicialización adicional. El funcionamiento correcto fue verificado con la herramienta *Gamepad Tester*, que confirmó la lectura en tiempo real de los ejes analógicos y la identificación del dispositivo como **cafe-4004-Joystick VR**.

Además de la lectura de ejes, el firmware implementa la capacidad de **recepción de comandos de vibración** provenientes de Unity. A través de la misma interfaz USB HID, la aplicación puede enviar un comando de activación del motor háptico incorporado en el joystick, el cual se utiliza para alertar al operador sobre la proximidad del efector final a un objeto antes del contacto físico.

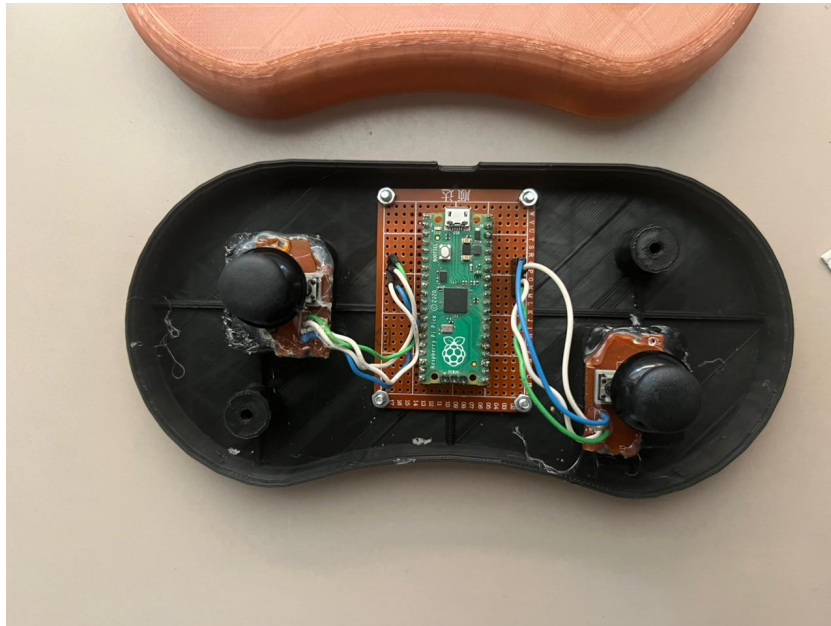


Figura 3: Interior del joystick: RPi Pico montado sobre placa de prototipado con los módulos analógicos de cada eje.



Figura 4: Vista exterior del prototipo con carcasa impresa en 3D y thumbsticks montados.

El estado actual del prototipo es funcional para todas las operaciones requeridas por la aplicación: lectura de los dos ejes analógicos, detección de botones y retroalimentación háptica. Las iteraciones futuras contemplan el refinamiento de la zona muerta (*deadband*), la aplicación de fil-

trado de ruido en los ADC y mejoras ergonómicas adicionales sobre la carcasa.

4 Desarrollo en Unity

La aplicación de entrenamiento fue desarrollada en **Unity 6000.3 LTS** utilizando el paquete de robótica **Flange** (`com.prelly.flange`) para la simulación cinemática del brazo robótico. Flange provee el modelo 3D articulado del **KUKA KR210 R3100-2**, un solver de cinemática inversa analítica integrado y una interfaz de control accesible desde C#, lo que evita la necesidad de implementar el cálculo cinemático desde cero. El joystick se integra al motor a través del *Unity Input System*, que reconoce el dispositivo como un gamepad HID estándar y expone sus ejes y botones mediante un *Input Action Asset* configurable desde el editor.



Figura 5: Escena de simulación en Unity: brazo KUKA KR210 con efector final tipo gancho integrado mediante el paquete *Flange*. Al fondo se aprecia el rack de ensamblaje y otros elementos del entorno industrial virtual.

4.1 Listado de Comandos y Canvas

La interfaz de usuario en pantalla (*Canvas*) fue implementada utilizando el sistema UI de Unity y permanece visible durante toda la sesión de entrenamiento. El panel se organiza en dos bloques principales, visibles en la Figura 6: el bloque de **Acciones de Control (PID)**, que muestra en tiempo real los valores de acción de control ($^{\circ}/s^2$) calculados por el controlador PID de cada articulación (J1–J6), y la **Guía de Controles (PS4)**, que lista el mapeo de botones y palancas del joystick para referencia inmediata del operador. En la esquina inferior izquierda se presenta la **cámara del gripper**, una sub-ventana que muestra la vista en tiempo real desde una cámara montada sobre el efector final, facilitando el posicionamiento preciso de la garra sobre las celdas de batería.



Figura 6: Panel de interfaz (*Canvas*) en ejecución: bloque de **Acciones de Control (PID)** con las acciones calculadas por articulación (J1–J6), guía de comandos del joystick PS4 y vista en tiempo real de la cámara embebida en el gripper (esquina inferior izquierda).

Las acciones de entrada se definen en el *Input Action Asset* `PS4Joystick.inputactions`. La Tabla 1 resume el mapeo físico completo de las 16 acciones configuradas en el asset. El sistema opera en dos modos exclusivos, conmutados mediante la acción `ModoCamara` (L3): en **modo Robot** (`IsCameraMode = false`), los analógicos mueven el *Tool Center Point* (TCP) del manipulador en

el espacio cartesiano; en **modo Cámara** (`IsCameraMode = true`), los mismos analógicos controlan la posición y orientación de la cámara principal al estilo *first-person*. Al tratarse de una propiedad estática en `JoystickAdapter`, el componente `CameraJoystickController` —adjunto a la *Main Camera*— puede consultarla sin requerir una referencia explícita entre objetos.

Tabla 1: Mapeo completo de acciones de entrada (`PS4Joystick.inputactions`).

Acción	Tipo	Control físico	Uso
<i>Modo Robot – movimiento del TCP</i>			
<code>MoveX</code>	1D Axis	L-Stick Left/Right	Traslación lateral del TCP
<code>MoveY</code>	1D Axis	R-Stick Up/Down	Traslación vertical del TCP
<code>MoveZ</code>	1D Axis	L-Stick Up/Down	Traslación frontal del TCP
<i>Modo Robot – control de articulación J6 y garra</i>			
<code>J6AntiHor</code>	Button	L1 (leftShoulder)	Rotación de J6 antihorario
<code>J6Hor</code>	Button	R1 (rightShoulder)	Rotación de J6 horario
<code>J6Home</code>	Button	Cuadrado (buttonWest)	Homing de J6 a posición cero
<code>Gripper</code>	Button	R3 (rightStickPress)	Toggle apertura/cierre de garra
<i>Modo Cámara – navegación de la cámara principal</i>			
<code>MoveForward</code>	1D Axis	L-Stick Up/Down	Avance/retroceso de la cámara
<code>MoveSide</code>	1D Axis	L-Stick Left/Right	Desplazamiento lateral de la cámara
<code>ViewUp</code>	1D Axis	R-Stick Up/Down	Rotación vertical (pitch)
<code>ViewSide</code>	1D Axis	R-Stick Left/Right	Rotación horizontal (yaw)
<i>Acciones globales</i>			
<code>ModoCamara</code>	Button	L3 (leftStickPress)	Toggle modo Robot / Cámara
<code>Click</code>	Button	X/Cruz (buttonSouth)	Confirmación / selección
<code>Pause</code>	Button	Options (start)	Pausar simulación
<code>SelectorUP / DOWN</code>	Button	D-pad Up/Down	Navegación de perfil de control
<code>SelectorLEFT / RIGHT</code>	Button	D-pad Left/Right	Cambio de escena/modo

En modo Robot, el componente `JoystickAdapter` construye en cada `FixedUpdate` el incremento de posición del TCP en espacio mundo:

$$\Delta \mathbf{p} = (\hat{\mathbf{d}}_X \cdot u_X + \hat{\mathbf{Y}}_{\text{mundo}} \cdot u_Y + \hat{\mathbf{d}}_Z \cdot u_Z) \cdot v \cdot \lambda_{\text{payload}} \cdot \Delta t \quad (1)$$

donde $u_X, u_Y, u_Z \in [-1, 1]$ son los valores leídos de las acciones `MoveX`, `MoveY` y `MoveZ`; v es la velocidad máxima configurada en el Inspector; $\hat{\mathbf{d}}_X$ y $\hat{\mathbf{d}}_Z$ son los versores mundo asignados por el remapeo geométrico (Sección 4.2) con signo fijo +1; y $\lambda_{\text{payload}} \in (0, 1]$ es un factor de penalización proporcional a la masa del objeto agarrado —implementado en `JoystickAdapter` como `payloadSpeedMultiplier`— que reduce la velocidad cartesiana de la consigna conforme aumenta

la carga, produciendo el efecto perceptible de “carga pesada = movimiento más lento” en VR. El incremento se aplica sobre la pose actual del TCP y se pasa al solver de cinemática inversa de Flange; si la solución es válida, se actualizan las consignas articulares.

4.2 Remapeo Geométrico de Direcciones

Cuando el operador navega libremente por la escena en modo Cámara y luego retoma el control del robot, la dirección “al frente” del punto de vista puede no coincidir con ningún eje de movimiento predefinido. Para que los analógicos siempre empujen el TCP en la dirección intuitiva —aquella que apunta directamente del operador hacia el efector— se implementó el método `RemapAxesFromCamera()` en `JoystickAdapter`.

El algoritmo calcula primero el vector $\vec{C}$ desde la posición de la cámara hasta la posición del TCP, anula su componente vertical y lo normaliza para obtener una dirección puramente horizontal en el plano XZ del mundo:

$$\hat{\mathbf{d}}_Z = \frac{(\mathbf{p}_{\text{TCP}} - \mathbf{p}_{\text{cam}})|_{Y=0}}{\|(\mathbf{p}_{\text{TCP}} - \mathbf{p}_{\text{cam}})|_{Y=0}\|} \quad (2)$$

Este versor se asigna directamente a la dirección de avance de la palanca `MoveZ` con signo positivo fijo. La dirección lateral de `MoveX` se obtiene como el producto vectorial entre el eje $+Y$ del mundo y $\hat{\mathbf{d}}_Z$:

$$\hat{\mathbf{d}}_X = \frac{\hat{\mathbf{Y}}_{\text{mundo}} \times \hat{\mathbf{d}}_Z}{\|\hat{\mathbf{Y}}_{\text{mundo}} \times \hat{\mathbf{d}}_Z\|} \quad (3)$$

también con signo positivo fijo. De este modo, empujar el analógico izquierdo hacia adelante siempre mueve el TCP en la dirección que apunta hacia él desde la cámara, y empujarlo a la derecha lo desplaza a su derecha geométrica, con independencia de la orientación del efector o de la posición de la cámara en la escena.

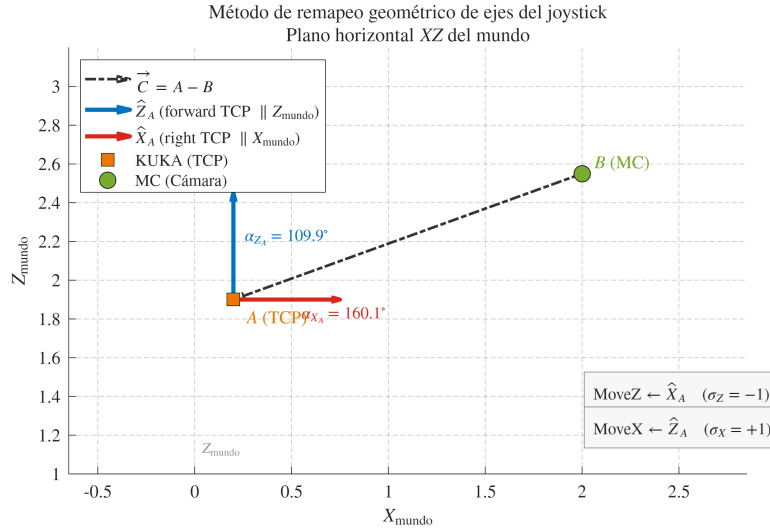


Figura 7: Plano horizontal XZ del mundo ilustrando el remapeo: el vector $\hat{\mathbf{d}}_Z$ apunta de la cámara (MC) al TCP del manipulador (KUKA), y $\hat{\mathbf{d}}_X$ se obtiene como su perpendicular en el plano horizontal.

4.3 Control PID en Articulaciones

Cuando el solver de Flange devuelve una solución IK válida, `JoystickAdapter` no aplica los ángulos resultantes directamente sobre el `MechanicalGroup`: el `JointTarget` obtenido se pasa como consigna al método `ApplyPID()`, que calcula para cada una de las seis articulaciones un torque virtual, lo convierte en una aceleración angular ponderada por la inercia efectiva del eslabón y finalmente integra posición y velocidad respetando límites físicos y de seguridad.

La inercia efectiva de cada articulación se calcula en `RobotDynamics.ComputeEffectiveInertia()` a partir de las masas y el centro de masa (CoM) de cada eslabón del KUKA KR210, tomados del modelo URDF utilizado en el curso *Robotics Nanodegree* (RoboND) de Udacity y hardcodeados en el arreglo estático `RobotDynamics.Links`: 138 kg para `link_1`, 95 kg para `link_2`, 71 kg para `link_3`, 17 kg para `link_4`, 7 kg para `link_5` y 0.5 kg para `link_6`, cada uno con su CoM expresado en el espacio local del `TransformJoint` correspondiente. Para la articulación i , la inercia efectiva se obtiene como

$$J_{\text{eff}}[i] = \sum_{j \geq i} m_j \cdot d_{ij}^2 \quad (4)$$

donde d_{ij} es la distancia perpendicular entre el eje de giro de la articulación i en el mundo y el

CoM (también en mundo) del eslabón j . Si el gripper sostiene un objeto, `ComputeEffectiveInertia()` acepta opcionalmente la masa y posición mundial del payload, obtenidos de `GripperController.GrabbedMass` y `GripperController.GrabbedWorldPosition`, sumándolos a $J_{\text{eff}}[i]$ de cada articulación como una masa puntual ubicada en la posición mundial del `graspPoint`.

El controlador de cada articulación es una instancia de `JointPID`, con ganancias base configurables en el Inspector de `JoystickAdapter` (`_kpBase = 20`, `_kiBase = 1`, `_kdBase = 0.5`, en unidades de $(^\circ/s^2)$ por $^\circ$, $(^\circ \cdot s)$ y $(^\circ/s)$ respectivamente, a una inercia de referencia `_referenceInertia = 100 kg·m2`). El término derivativo no se calcula sobre la derivada clásica del error sino como un error de velocidad entre la velocidad de la consigna IK y la velocidad articular medida:

$$\tau[k] = K_p e[k] + K_i \sum e[k] \Delta t + K_d (\dot{q}_{\text{target}}[k] - \dot{q}_{\text{medida}}[k]) \quad (5)$$

con $e[k] = \text{Mathf.DeltaAngle}(q_{\text{actual}}, q_{\text{target}})$, término integral saturado en $\pm 200^\circ \cdot s$ (anti-windup), y $\dot{q}_{\text{medida}}$ estimada como $\text{DeltaAngle}(q_{\text{previo}}, q_{\text{actual}})/\Delta t$. La velocidad de consigna $\dot{q}_{\text{target}}$ se estima en `JoystickAdapter` a partir de la diferencia entre el `JointTarget` IK del tick actual y el anterior, y alimenta además un término de *feedforward* que compensa el amortiguamiento viscoso discreto, garantizando que el seguimiento de velocidad no dependa únicamente de la ganancia derivativa. El torque resultante se divide por la inercia normalizada $j_{\text{norm}} = \max(J_{\text{eff}}[i]/\text{referenceInertia}, 0.05)$ —con un piso del 5% para evitar inestabilidad numérica en los eslabones livianos de la muñeca— para obtener la aceleración angular, que se satura en $\pm 720^\circ/s^2$. La velocidad se integra, se le aplica un amortiguamiento viscoso (`_velocityDamping = 5 s-1`) y se satura en $\pm 360^\circ/s$. La posición articular resultante se recorta contra los límites físicos de cada articulación; si un límite se alcanza, la velocidad se frena a cero y el integrador del PID correspondiente se reinicia para evitar windup. El resultado final se aplica mediante `MechanicalGroup.SetJoints()`, siempre como ángulos articulares en grados.

4.4 Movimiento del Gripper y Realimentación Háptica

La apertura y cierre de la garra se controla con un único botón (acción `Gripper`, R3), gestionado por `Ctrl_OnRobotRG2_Custom`. Un clic corto alterna el estado `_isOpen` y dispara la animación del efector OnRobot RG2: la carrera (*stroke*, 0–100 mm) se convierte en un ángulo de servo mediante un polinomio de ajuste (`Polyval`) y se interpola con `Mathf.MoveTowards` a una velocidad de 180 mm/s, rotando físicamente los seis `Rigidbody` de los dedos mediante `MoveRotation`. Mantener el botón presionado un segundo, en cambio, no cierra la garra sino que dispara un *homing* de la articulación J6 a su cero físico de 17.7° (`JoystickAdapter.ResetJ6ToZero()`), la misma acción asociada al botón Cuadrado (`J6Home`).

La lógica de agarre vive en `GripperController` y no depende únicamente de un tag ni de

un simple raycast: cada dedo tiene un `GripperTriggerForwarder` sobre su cara interna que detecta por *trigger* los objetos con tag "Agarrable" y notifica los contactos a `GripperController.NotifyFingerContact()`. Un objeto solo se agarra si existe una ventana de cierre explícita (`isClosing`) y si el objeto registra contacto simultáneo de ambos dedos (`HasOpposingInnerContacts()`); tocar un objeto con la garra ya cerrada nunca lo adhiere. Al agarrarlo, `GrabObject()` guarda la masa original del `Rigidbody`, la suma a la masa del `Rigidbody` del gripper, vuelve cinemático el objeto agarrado y lo emparenta al `graspPoint` para lograr un movimiento 1:1 sin deslizamientos. Al soltar, se restauran masa y físicas, y `MoveGrabbedObjectToSafeReleaseHeight()` corrige la posición si la suelta ocurriera por debajo del piso o de la altura mínima de recogida.

La proximidad para la realimentación háptica se mide con `GripperDistanceSensor`, que emplea un muestreo cónico en lugar de un único raycast: dispara varios `Physics.OverlapSphereNonAlloc` (8 muestras por defecto) a lo largo de la dirección local del sensor, con una apertura de 22.5° de semiángulo y hasta 0.4m de distancia máxima, ignorando la propia jerarquía del gripper. La distancia mínima detectada define si el agarre es seguro (`IsGripDistanceSafe`, entre 0.01 y 0.05 m) y retroalimenta al operador mediante texto en el Canvas. Independientemente de ese umbral de seguridad, cuando no hay ningún objeto agarrado y la distancia detectada cae por debajo de `vibrationStartDistance` (0.15m, con histéresis hasta 0.18m para desactivar), el sensor activa la vibración del joystick a través de `JoystickVibrationHidOutput.SetMotor(true)`.

El envío del comando de vibración desde Unity al joystick RPi Pico opera por dos caminos en paralelo dentro de `JoystickVibrationHidOutput`. El principal es una escritura HID directa por Win32 (`CreateFile/WriteFile/HidD_SetOutputReport` sobre `hid.dll` y `setupapi.dll`), localizando el dispositivo por su VID/PID (0xCAFE/0x4004) y enviando un reporte de salida de dos bytes: un *report ID* (0x00) seguido de un valor de motor (0x01 encendido, 0x00 apagado). En paralelo, si hay un gamepad PS4 conectado a través del Input System (`DualShockGamepad`), se replica el estado del motor como *rumble* nativo (`SetMotorSpeeds`), lo que permite sustituir el joystick RPi Pico por un control PS4 estándar durante sesiones de prueba sin hardware.

La cámara del gripper se implementa como una `Camera` independiente con `targetTexture` apuntando a un `RenderTargetTexture` dedicado, mostrado en el Canvas mediante una `RawImage` en la esquina inferior izquierda. Su posición y orientación se actualizan en `GripperTopCameraFollow.LateUpdate()`: la cámara sigue al efector final desde arriba y se mantiene siempre mirando hacia abajo, con el vector *up* proyectado desde el *forward* del efector sobre el plano horizontal, de modo que la vista gira solidariamente con J6.

4.5 Comunicación UDP con Arquitectura Servidor–Cliente

La aplicación Unity actúa como **servidor UDP** con arquitectura de suscripción dinámica, implementada en el script `JointStateBroadcaster.cs`. A diferencia de un esquema de destino fijo, el

servidor mantiene una lista de suscriptores activos: cualquier cliente que envíe un datagrama de suscripción ("HELLO") a la IP y puerto de Unity queda registrado y comienza a recibir el estado articular del robot. El servidor soporta opcionalmente la eliminación automática de suscriptores inactivos (`_autoCleanupEnabled`, desactivado por defecto) tras un tiempo de inactividad configurable (`_subscriberTimeoutSeconds = 15 s`); cuando está activado, los clientes que dejan de enviar *keep-alive* periódicos son removidos de la lista. Esto permite que múltiples clientes —un script de Python, una visualización en MATLAB, un dashboard web en el celular— operen simultáneamente sin ninguna reconfiguración del servidor.

El servidor transmite el estado articular a 25 Hz como un mensaje ASCII por paquete con el formato "`q1,q2,q3,q4,q5,q6`", donde los seis valores corresponden a los ángulos articulares en grados. Se especifica `CultureInfo.InvariantCulture` en la serialización para garantizar el uso del punto decimal, evitando conflictos con la configuración regional del sistema operativo.

Se desarrollaron cuatro clientes de referencia que comparten el mismo protocolo de suscripción y reutilizan la misma tabla de parámetros DH y la misma fórmula de calibración mecánica (`JointConfig.Offset/Factor`).

El primero, `udp_joint_receiver.py`, es el cliente de consola más simple: solicita por consola la IP y puerto del servidor, envía el datagrama de suscripción inicial y mantiene un hilo auxiliar de *keep-alive* que repite el "HELLO" cada 5 s. Al recibir cada paquete valida que contenga exactamente 6 campos numéricos e imprime los ángulos con marca de tiempo.

El segundo, `udp_robot_viewer.py`, añade visualización 3D desde Python utilizando el *Robotics Toolbox for Python* de Peter Corke (`roboticstoolbox-python`) con el backend `pyplot` (matplotlib). El backend *Swift* (three.js) fue descartado porque requiere geometría STL/URDF; el modelo puramente cinemático (DHRobot) construido a partir de los parámetros DH solo es graficable con `pyplot`, que dibuja el robot como figura de palillos (*stick figure*) a partir de la cinemática directa, igual que MATLAB. El script drena el buffer de recepción en cada ciclo para procesar únicamente el paquete más reciente y evitar el lag acumulado, y superpone en la figura un panel de texto con los seis ángulos articulares actuales en grados, tal como llegan de Unity. La Figura 8 muestra el resultado.

El tercero, `robot_udp_plot.m`, reproduce el mismo mecanismo de suscripción en MATLAB y anima el modelo cinemático del KR210 con la librería *Robotics Toolbox* de Peter Corke (`SerialLink`), aplicando idéntica estrategia de drenado de buffer. La Figura 9 muestra la réplica resultante.

El cuarto es un **dashboard web** implementado en Python con servidor HTTP embebido (`udp_robot_dashboard.py`), accesible desde cualquier navegador en la misma red. La interfaz muestra el estado articular como gauges circulares para cada articulación (J1–J6), una vista esquemática 3D interactiva del robot y un indicador de paquetes recibidos. El mismo dashboard se ejecuta localmente en la PC (Figura 10) o se sirve a dispositivos móviles en la red Tailscale (Figura 11).

La comunicación se verifica sobre la red virtual **Tailscale**, que conecta mediante VPN el equipo

con Unity, el PC de MATLAB/Python y el teléfono, permitiendo operar los cuatro clientes simultáneamente con el servidor en ejecución.

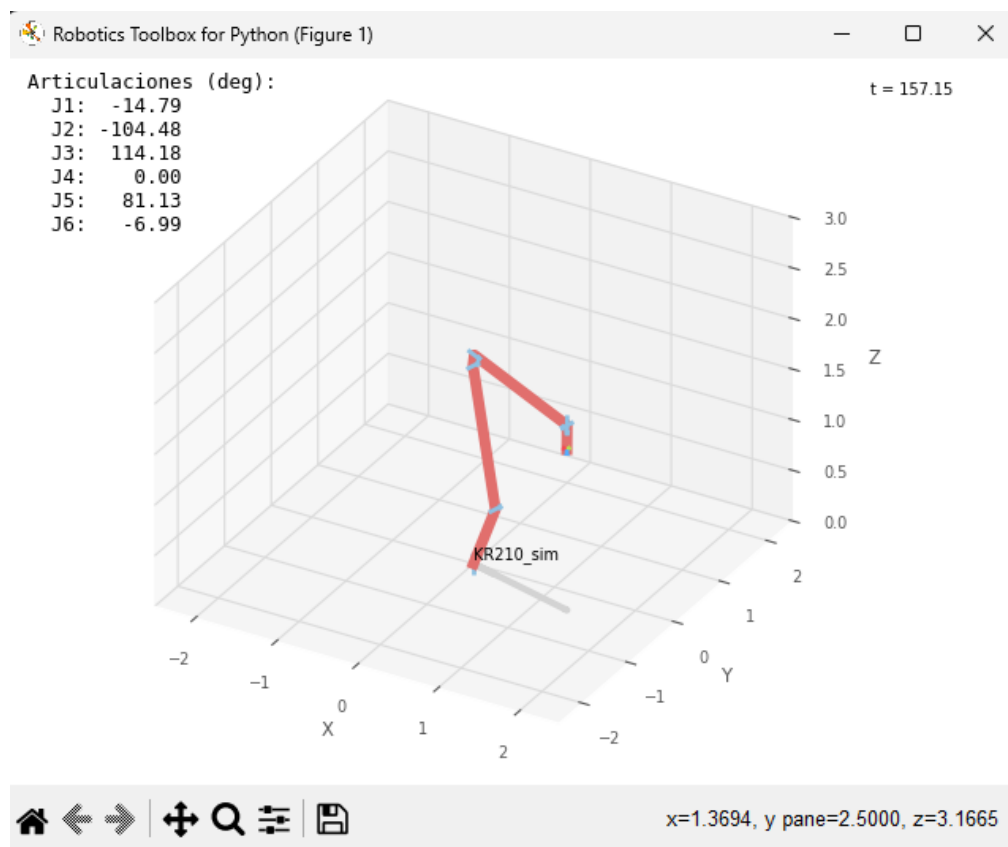


Figura 8: Visor 3D en Python: el script `udp_robot_viewer.py` anima el modelo cinemático del KR210 en tiempo real usando *Robotics Toolbox for Python* (backend `pyplot`) y muestra los ángulos articulares actuales recibidos vía UDP desde Unity.

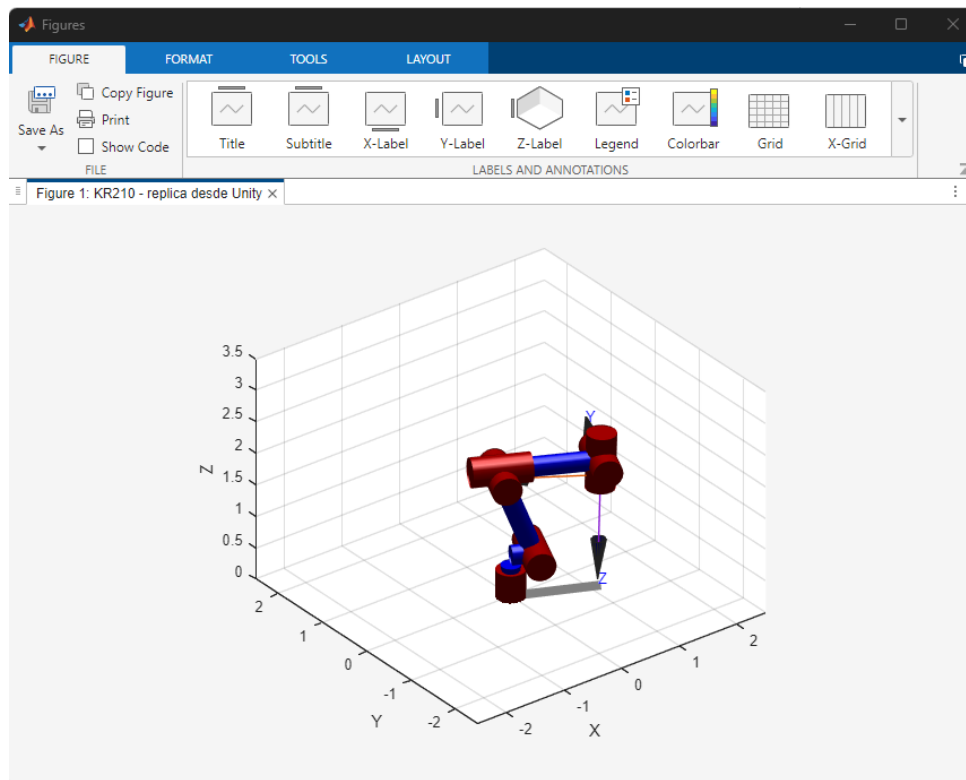


Figura 9: Réplica del estado articular del KR210 en MATLAB mediante el *Robotics Toolbox* de Peter Corke, recibida vía UDP desde Unity en tiempo real.

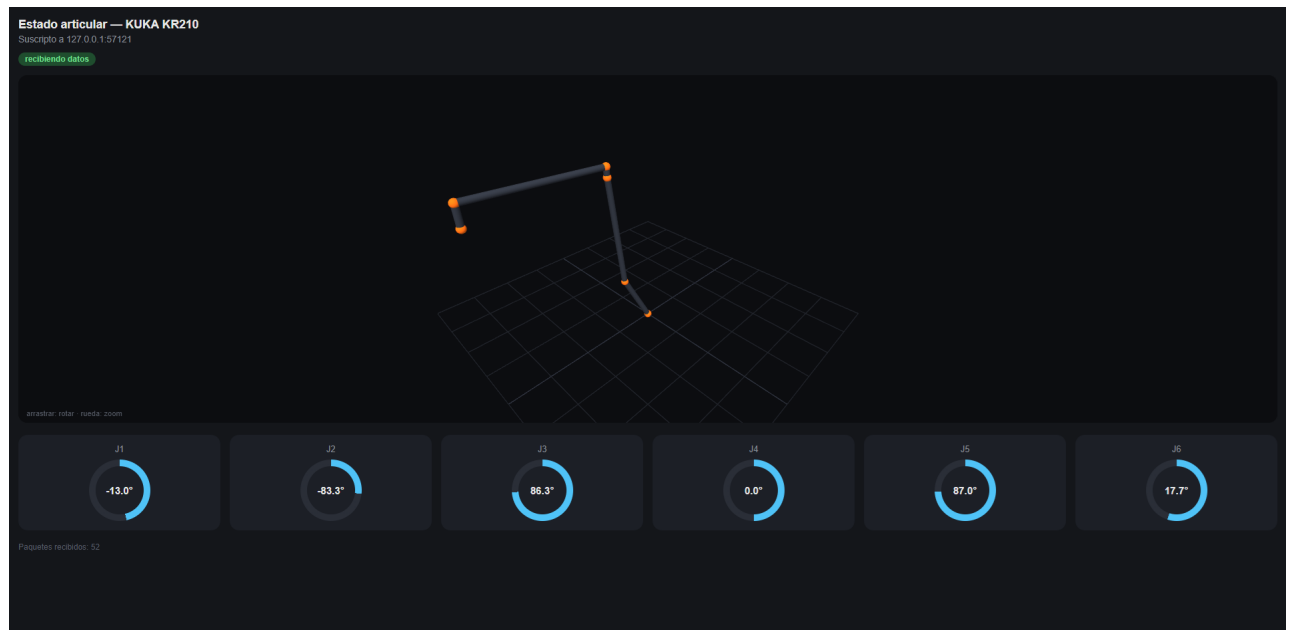


Figura 10: Dashboard web ejecutado en PC: muestra los ángulos articulares J1–J6 como gauges circulares, una vista esquemática 3D interactiva y el conteo de paquetes recibidos. Accesible desde cualquier navegador en la red Tailscale.

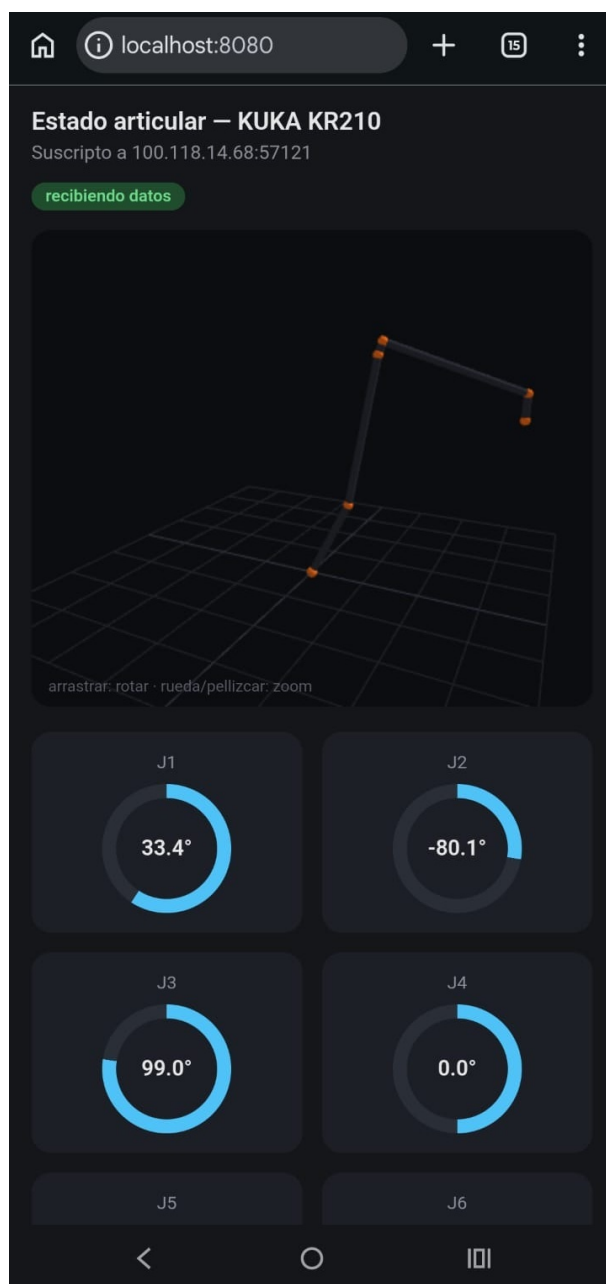


Figura 11: Dashboard web móvil: misma aplicación servida al teléfono vía Tailscale, con la vista 3D y los gauges articulares en formato vertical.